

Digital Electronics (II SEM B.Tech. CSE)

Couse Code: NECE102



Dr. Swati Rajput

Assistant Professor

Department of Electronics Engineering

Indian Institute of Technology (Indian School of Mines) Dhanbad

Digital Electronics: Key Topics at a Glance



Unit No.	Topics to be Covered	Lecture Hours	Learning Outcome
1.	Number System, Logic Gates, Boolean Algebra, NAND and NOR Implementation of Logic Functions, Determination of Boolean Functions from Digital Circuits.	08	Acquire an understanding of number system, basic logic gates and their applications to develop digital circuits and systems.
2.	Standard Representations of Logical Functions, SOP and POS form, Simplification of Logical Functions using K-map.	08	Acquire the knowledge to represent the logic functions in standard form and its minimization.
3.	Combinational Logic Circuit: Adder, Subtractor, Magnitude Comparator, Code Converters etc. MSI Circuits: Multiplexers, Demultiplexers, Decoders, Encoders etc.	08	Understand the design and applications of combinational logic circuits.
4.	Sequential Circuits: Latches and Flip-flops, Counters, Shift Registers.	10	Acquire the ability to design and perform the analysis of different sequential circuits.
5.	Analysis of Sequential circuits: State tables and state diagrams, Timing Circuits	04	Acquire an understanding on the analysis of sequential circuits.
6.	Introduction to logic families, semiconductor memories, ROM, PAL, PLA and Microprocessor	04	Develop knowledge about the various techniques for IC designing for digital circuits.

Textbook:	Reference Books:
1. Floyd T.L., Digital Fundamentals, 10/2, Pearson Education, 2011.	1. Digital Logic and Computer Design by M. Morris Mano, Prentice –Hall, 2015. 2. Fundamental of Digital Electronics by A. Anand Kumar, 4/e Prentice Hall, 2016.

Digital Electronics: Lecture Plan



Sl. No.	Topics	No of classes (tentative)
1.	Introduction to Number Systems and Conversion Techniques	8
2.	Continue.....	
3.	1's and 2's complement	
4.	Logic Gates (AND, OR, NOT, NAND, NOR, XOR, XNOR)	
5.	Boolean Algebra: Laws and Theorems	
6.	Boolean Function Implementation Using NAND/NOR Gates	
7.	Determination of Boolean Functions from Digital Circuits	
8.	Continue.....	
9.	SOP Standard Forms of Logical Functions	8
10.	POS Standard Forms of Logical Functions	
11.	Simplification of Logical Functions Using Boolean Algebra	
12.	Introduction to Karnaugh Maps (K-Maps)	
13.	K-Map Simplification (2-variable Functions)	
14.	K-Map Simplification (3-variable Functions)	
15.	K-Map Simplification (4-variable Functions)	
16.	Don't Care Conditions in K-Map Simplification	

Digital Electronics: Lecture Plan

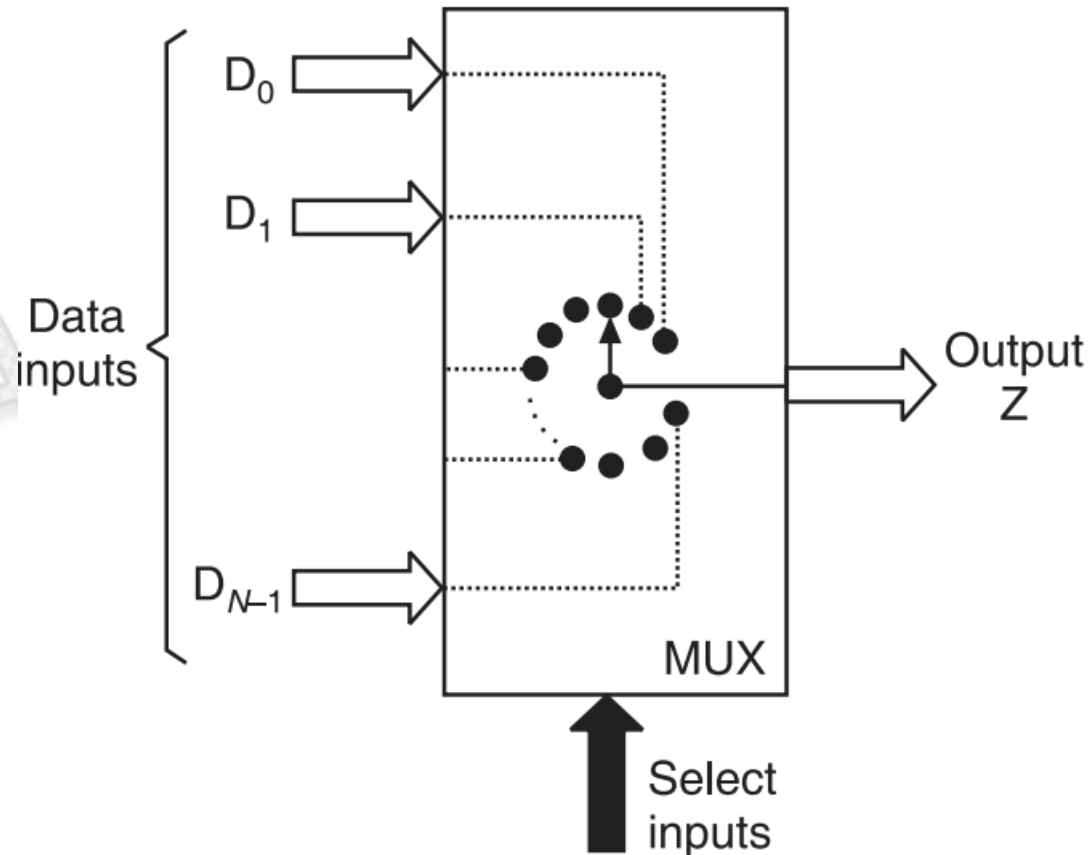


17.	Introduction to Combinational Logic Circuits	8
18.	Design of Adders (Half Adder, Full Adder)	
19.	Design of Subtractors (Half Subtractor, Full Subtractor)	
20.	Magnitude Comparator Design	
21.	Code Converters (Binary to Gray, Gray to Binary, BCD to Excess-3)	
22.	Multiplexers: Design and Applications	
23.	Demultiplexers: Design and Applications	
24.	Decoders and Encoders	
25.	Introduction to Sequential Circuits	10
26.	Latches: SR Latch, D Latch	
27.	Flip-Flops: SR, JK Flip-Flops	
28.	Flip-Flops: D and T Flip-Flops	
29.	Asynchronous Counters: Design and Operation	
30.	Continue.....	
31.	Synchronous Counters: Design and Operation	
32.	Continue.....	
33.	Shift Registers: Types and Applications	
34.	Continue.....	

Multiplexer



- ❖ A multiplexer (MUX) or data selector is a logic circuit that accepts several data inputs and allows only one of them at a time to get through to the output.
- ❖ The routing of the desired data input to the output is controlled by SELECT inputs.
- ❖ Normally there are 2^n input lines and n select lines whose bit combinations determine which input is selected.

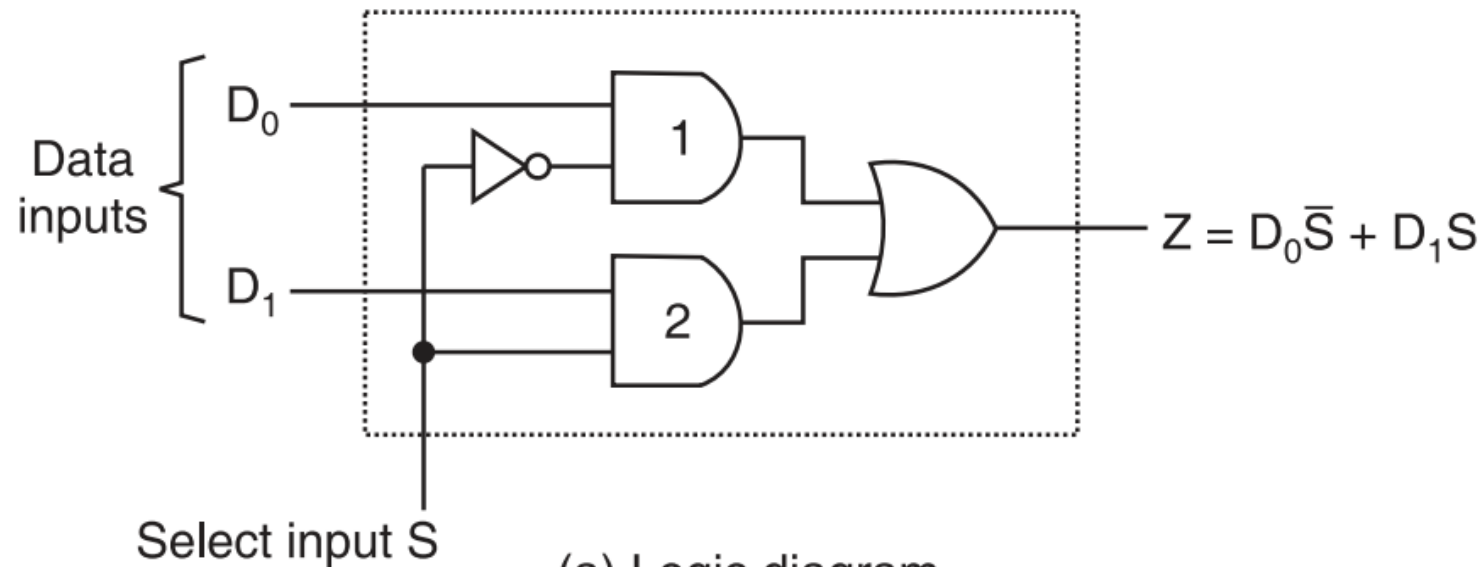


Basic 2 Input Multiplexer



- ❖ A 2-input multiplexer with data inputs D_0 and D_1 , and data select input S .
- ❖ It has two input lines; one data select line and one output line.

When $S = 0$, AND gate 1 is enabled and AND gate 2 is disabled. So, $Z = D_0$.
When $S = 1$, AND gate 1 is disabled and AND gate 2 is enabled. So, $Z = D_1$.



(a) Logic diagram

S	Output (Z)
0	D_0
1	D_1

(b) Function table

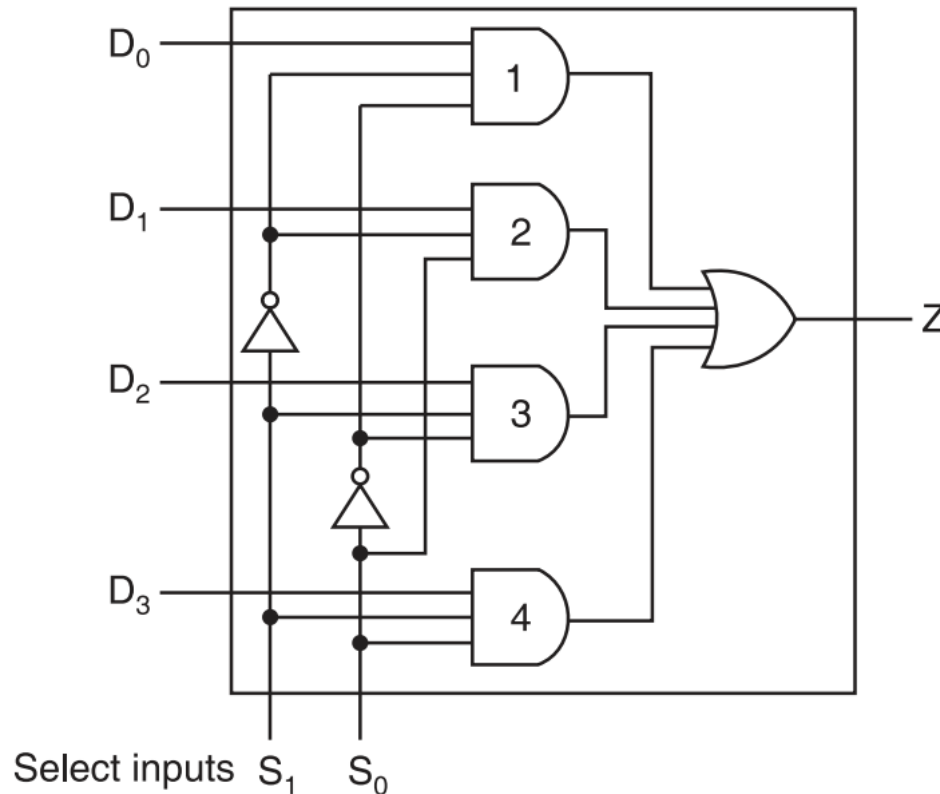
Figure 7.76 2-input multiplexer.

4 Input Multiplexer



- ❖ a 4-input multiplexer with data inputs D_0 , D_1 , D_2 , and D_3 , and data select inputs S_0 and S_1 . The logic levels applied to the S_0 and S_1 inputs determine which AND gate is enabled, so that its data input passes through the OR gate to the output.

$$Z = \bar{S}_1 \bar{S}_0 D_0 + \bar{S}_1 S_0 D_1 + S_1 \bar{S}_0 D_2 + S_1 S_0 D_3$$

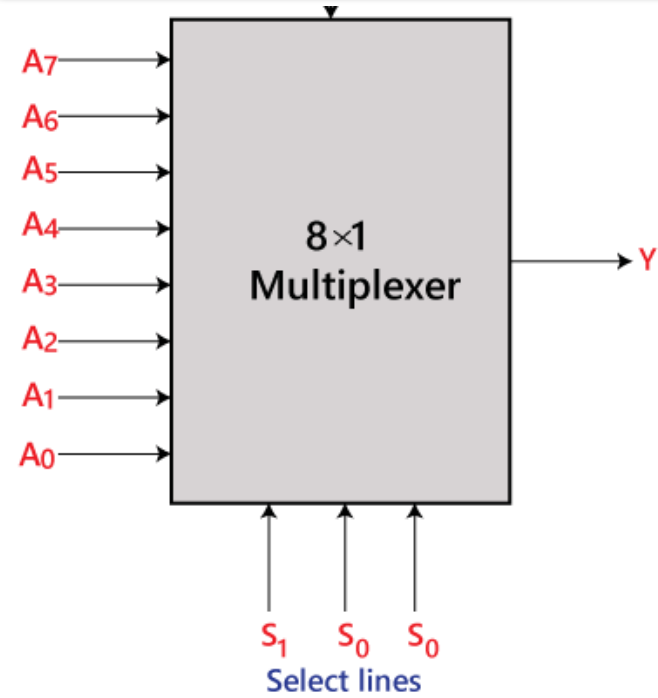


(a) Logic diagram

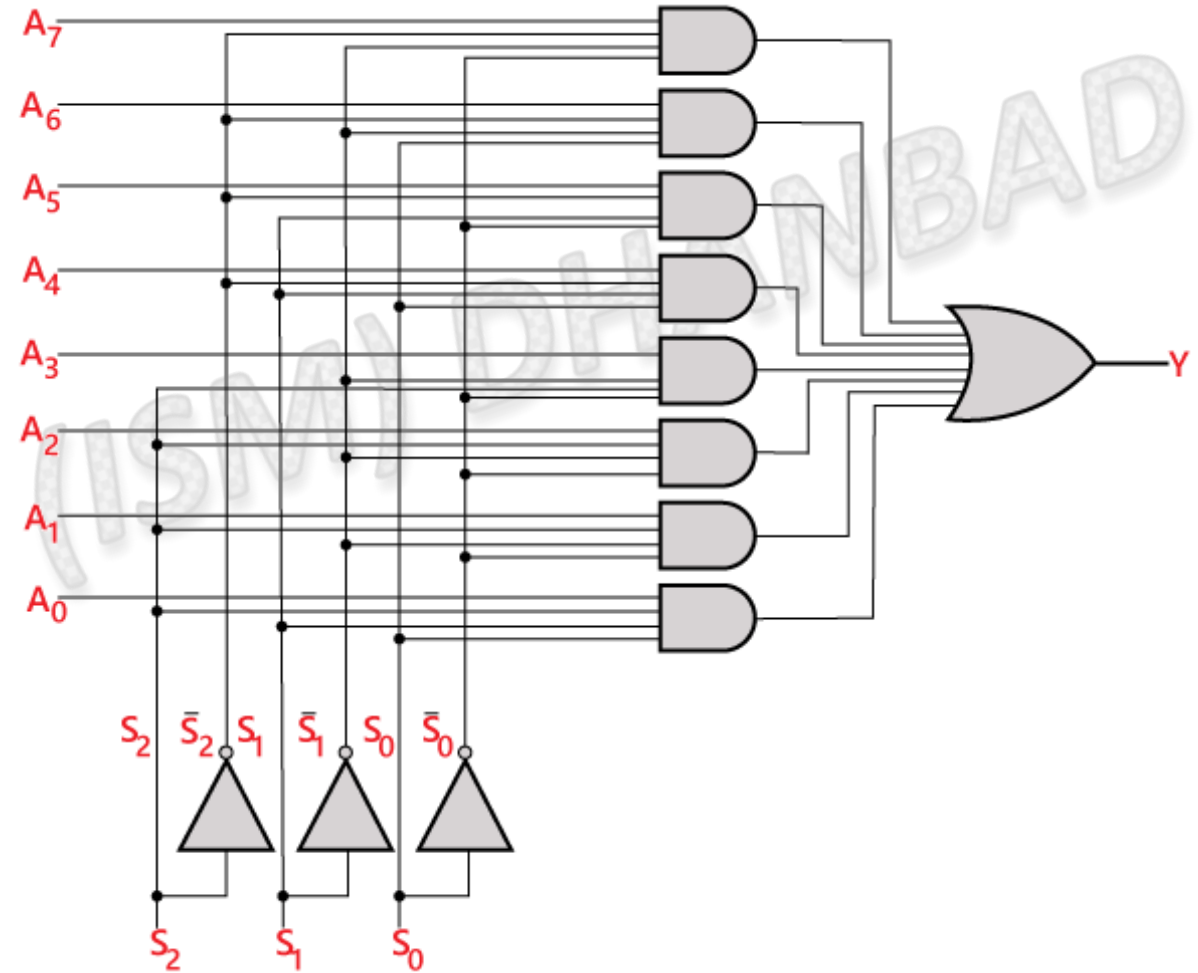
Select inputs		Output
S_1	S_0	Z
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

(b) Function table

8 Input Multiplexer



INPUTS			Output
S ₂	S ₁	S ₀	Y
0	0	0	A ₀
0	0	1	A ₁
0	1	0	A ₂
0	1	1	A ₃
1	0	0	A ₄
1	0	1	A ₅
1	1	0	A ₆
1	1	1	A ₇



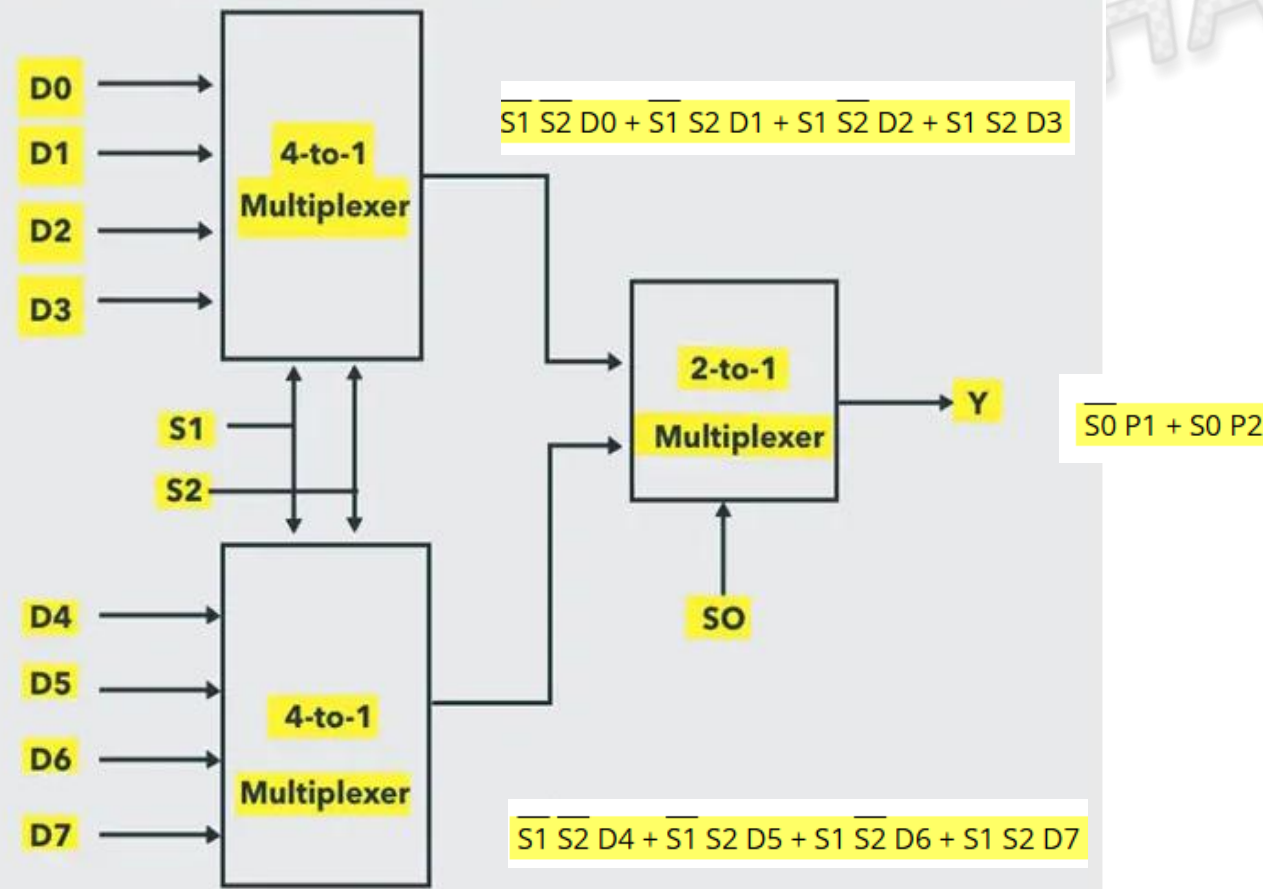
8 × 1 multiplexer using 4×1 and 2×1 multiplexer



$$Y = \overline{S_0} \overline{S_1} \overline{S_2} D_0 + \overline{S_0} \overline{S_1} \overline{S_2} D_1 + \overline{S_0} \overline{S_1} \overline{S_2} D_2 + \overline{S_0} \overline{S_1} \overline{S_2} D_3 + \overline{S_0} \overline{S_1} \overline{S_2} D_4 + \overline{S_0} \overline{S_1} \overline{S_2} D_5 + \overline{S_0} \overline{S_1} \overline{S_2} D_6 + \overline{S_0} \overline{S_1} \overline{S_2} D_7$$

$$Y = \overline{S_0} (\overline{S_1} \overline{S_2} D_0 + \overline{S_1} \overline{S_2} D_1 + \overline{S_1} \overline{S_2} D_2 + \overline{S_1} \overline{S_2} D_3) + \overline{S_0} (\overline{S_1} \overline{S_2} D_4 + \overline{S_1} \overline{S_2} D_5 + \overline{S_1} \overline{S_2} D_6 + \overline{S_1} \overline{S_2} D_7)$$

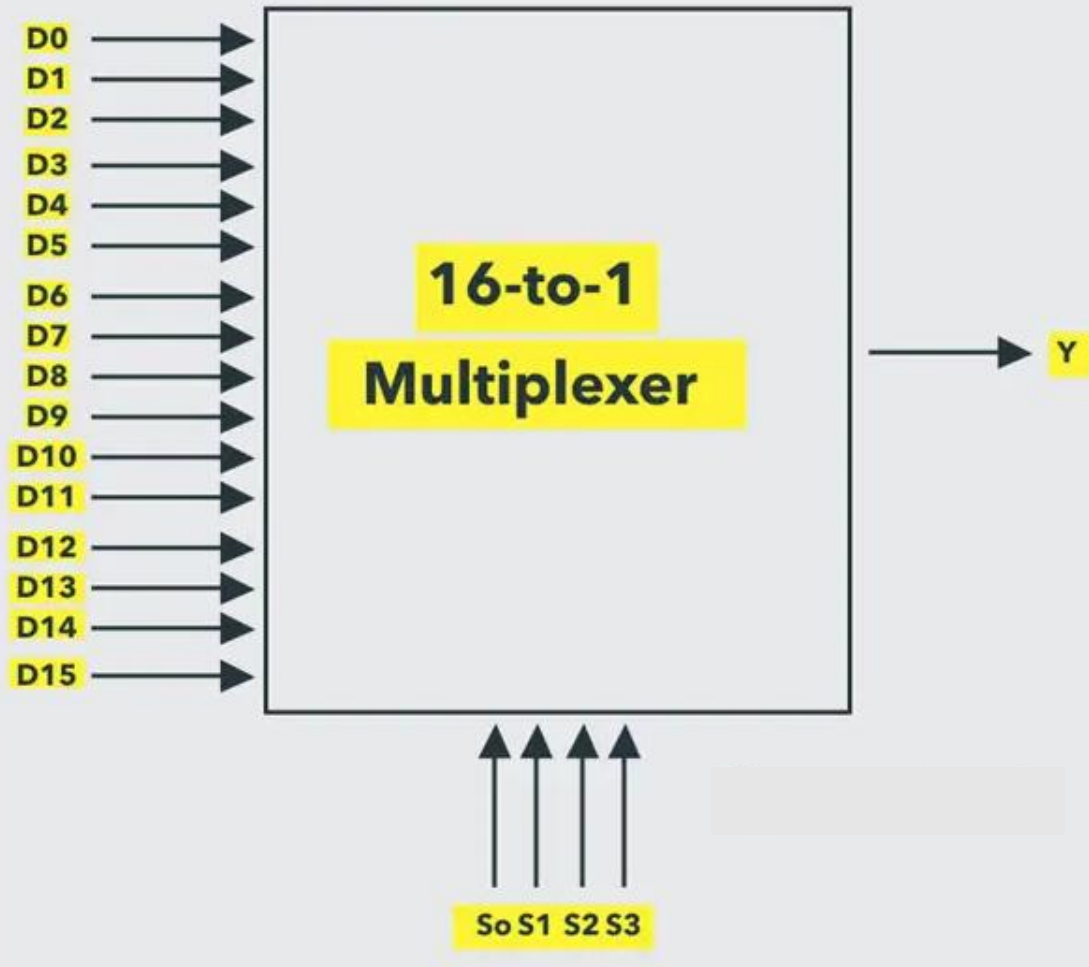
8-To-1 Mux Using Two 4-To-1 Muxes And One 2-To-1 Mux



16 to 1 Multiplexer



Block Diagram Of 16-To-1 Multiplexer

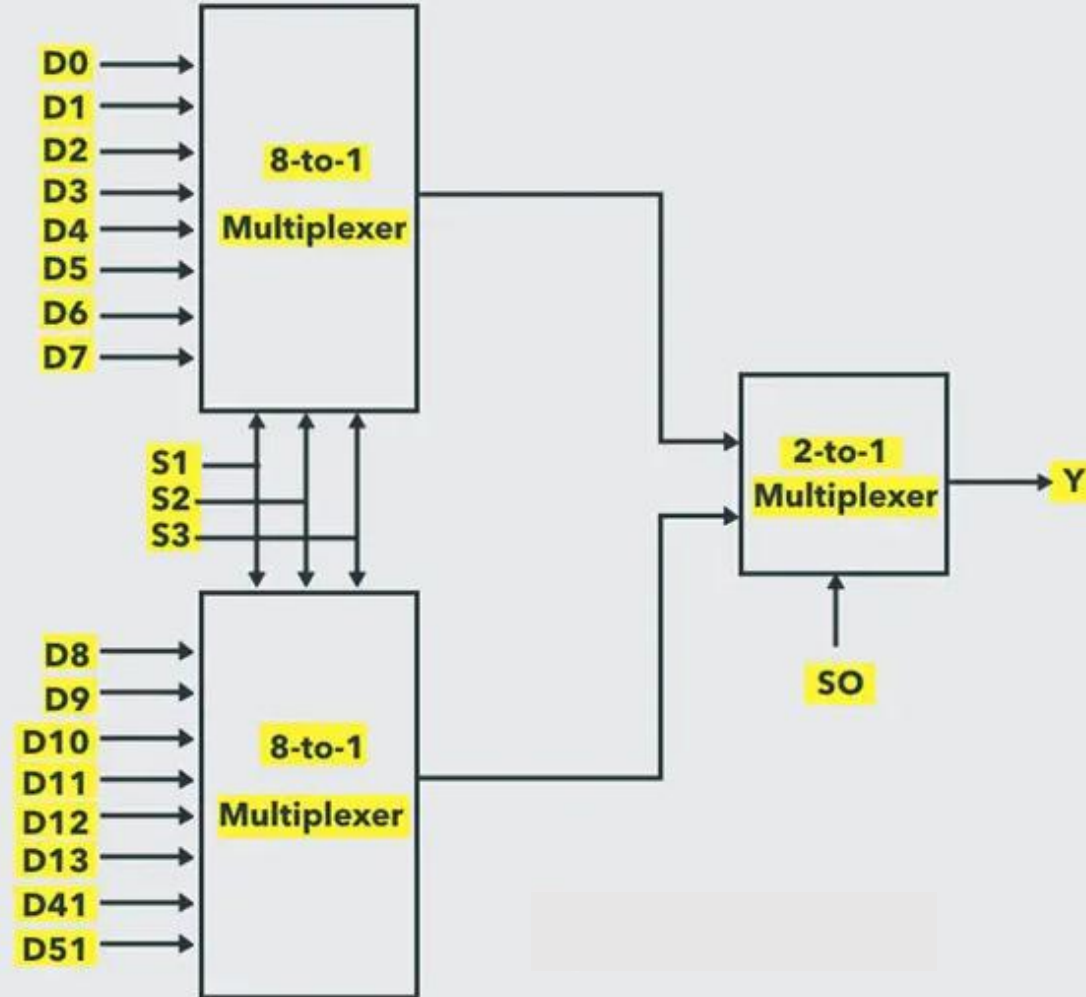


HANBAD

$$Y = \overline{S_0} \overline{S_1} \overline{S_2} \overline{S_3} D_0 + \overline{S_0} \overline{S_1} \overline{S_2} S_3 D_1 + \overline{S_0} \overline{S_1} S_2 \overline{S_3} D_2 + \overline{S_0} \overline{S_1} S_2 S_3 D_3 + \overline{S_0} S_1 \overline{S_2} \overline{S_3} D_4 + \overline{S_0} S_1 \overline{S_2} S_3 D_5 + \overline{S_0} S_1 S_2 \overline{S_3} D_6 + \overline{S_0} S_1 S_2 S_3 D_7 + S_0 \overline{S_1} \overline{S_2} \overline{S_3} D_8 + S_0 \overline{S_1} \overline{S_2} S_3 D_9 + S_0 \overline{S_1} S_2 \overline{S_3} D_{10} + S_0 \overline{S_1} S_2 S_3 D_{11} + S_0 S_1 \overline{S_2} \overline{S_3} D_{12} + S_0 S_1 \overline{S_2} S_3 D_{13} + S_0 S_1 S_2 \overline{S_3} D_{14} + S_0 S_1 S_2 S_3 D_{15}$$

16 × 1 multiplexer using 8 × 1 and 2 × 1 multiplexer

16-To-1 Mux Using Two 8-To-1 Mux And One 2-To-1 Mux



The 16-Input Multiplexer from Two 8-Input Multiplexers

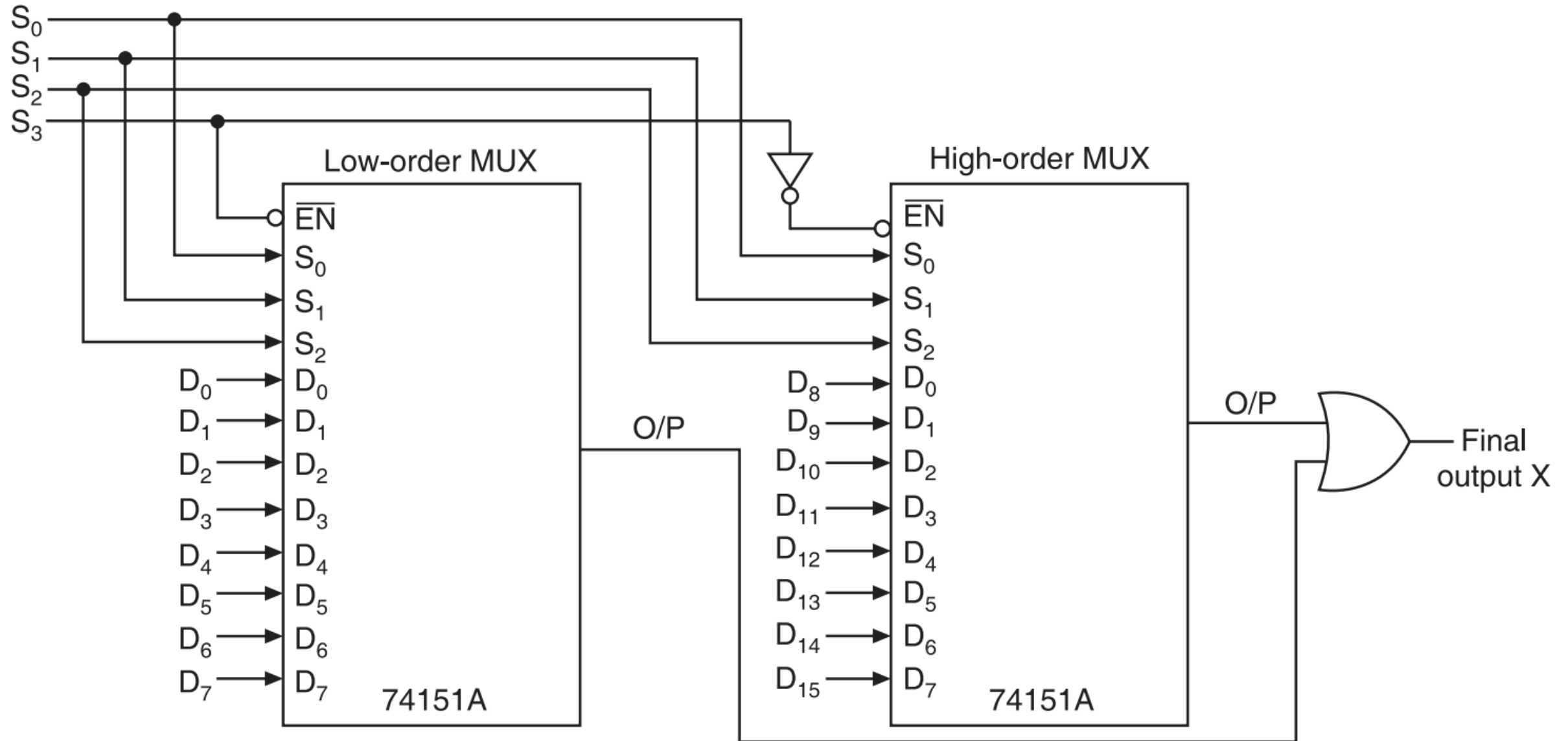


Figure 7.78 Logic diagram for cascading of two 8×1 muxes to get a 16-bit mux.

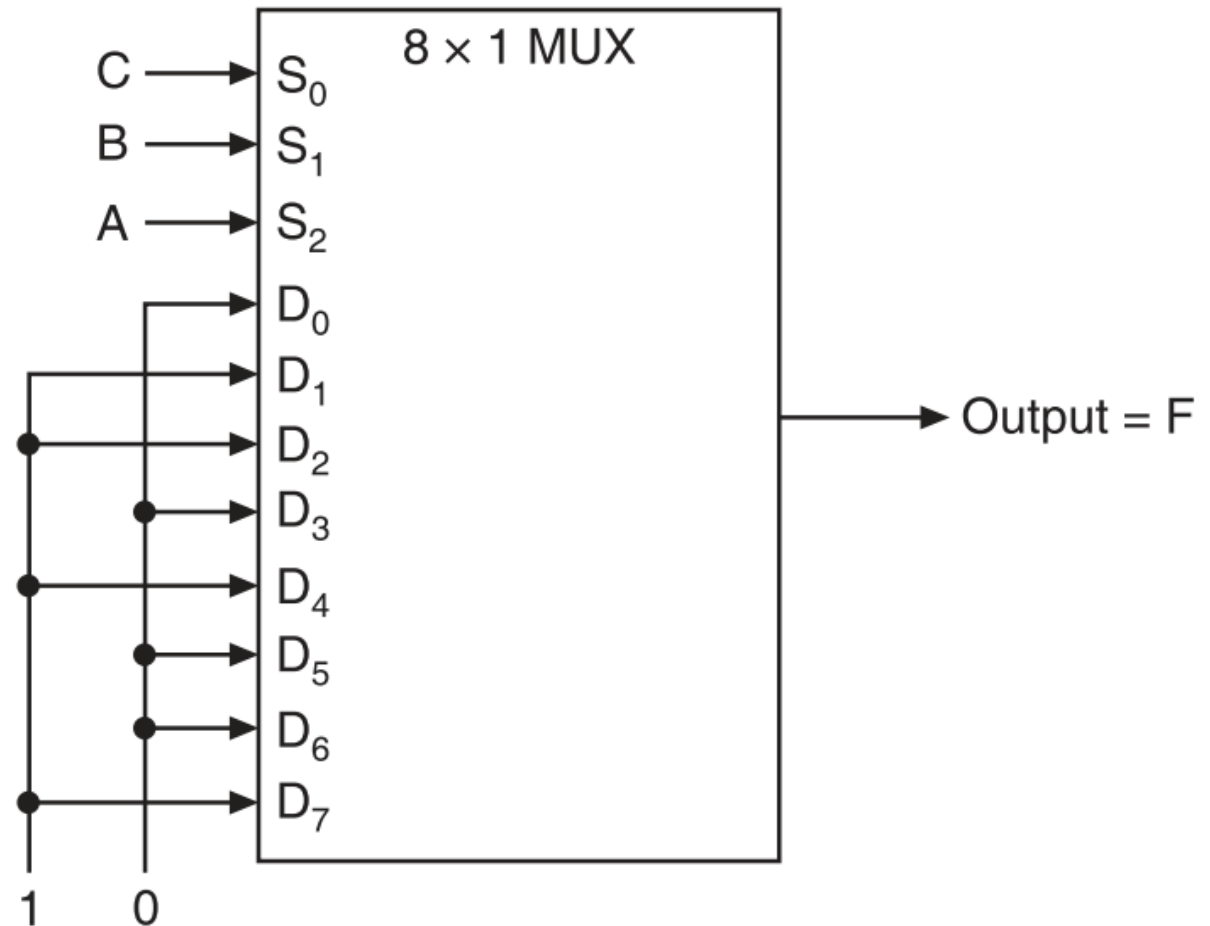
Multiplexer



Use a multiplexer to implement the logic function $F = A \oplus B \oplus C$.

S_2	S_1	S_0	$F = A \oplus B \oplus C$
A	B	C	
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(a) Truth table



(b) Logic diagram

Multiplexer

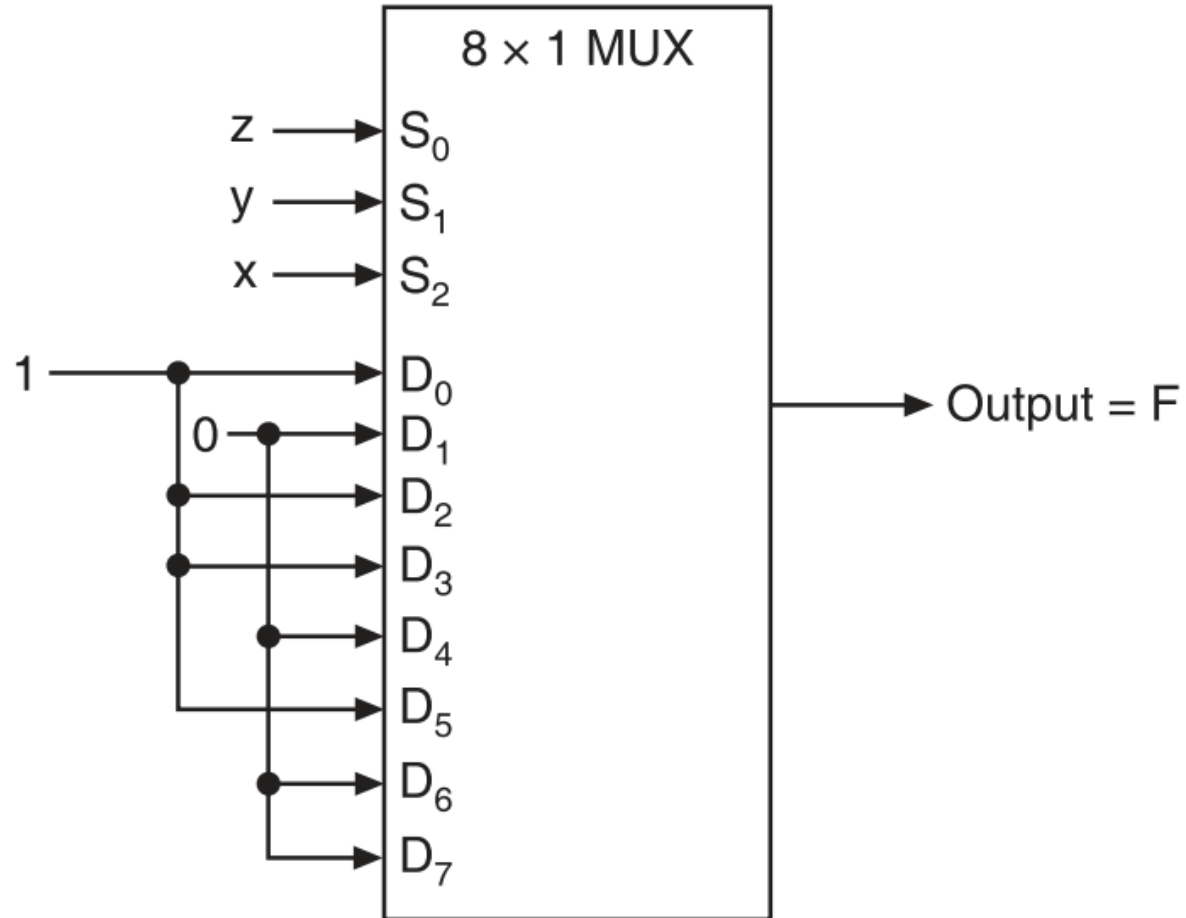


Implement the following function using 8-to-1 MUX

$$F(x, y, z) = \sum m(0, 2, 3, 5)$$

S_2	S_1	S_0	F
x	y	z	
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

(a) Truth table



(b) Logic diagram

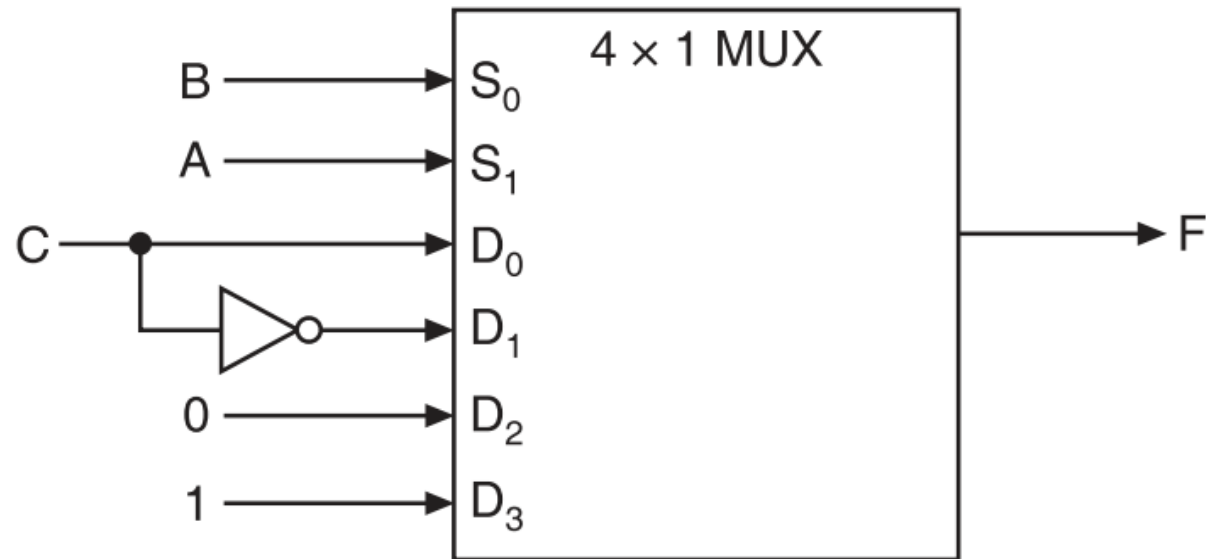
Multiplexer



Use a 4×1 MUX to implement the logic function
 $F(A, B, C) = \Sigma m(1, 2, 4, 7)$.

S_1	S_0		
A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

(a) Truth table



(b) Multiplexer implementation

Multiplexer



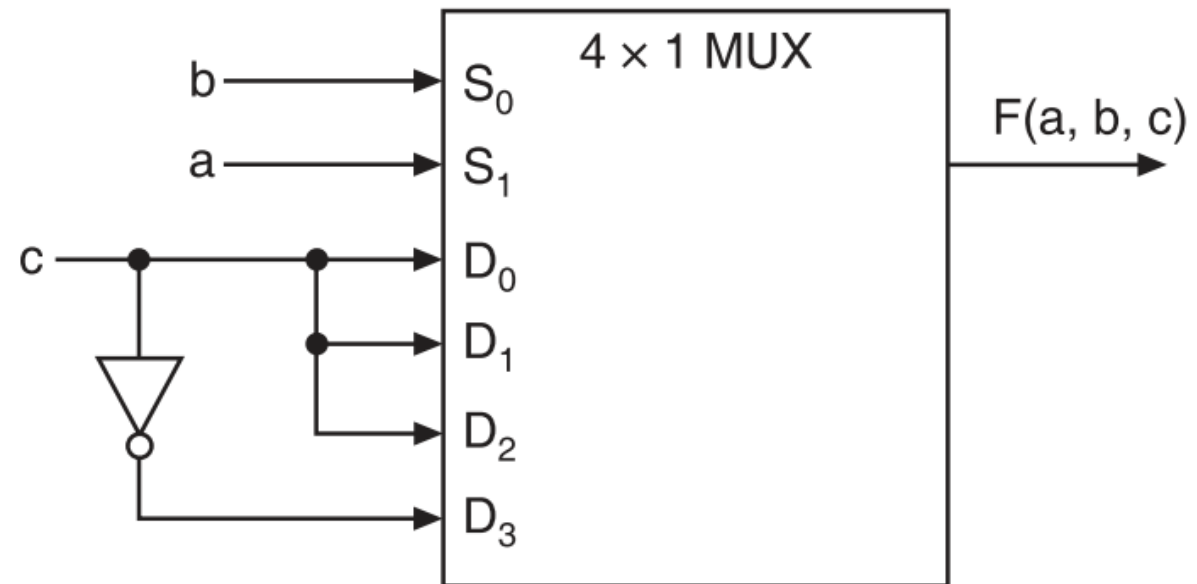
Implement the following function with a MUX:

$$F(a, b, c) = \sum m(1, 3, 5, 6) \quad \text{Choose } a \text{ and } b \text{ as select inputs.}$$

1. For both values of $ab = 00$, $ab = 01$ and $ab = 10$, $F = c$. So D_0 , D_1 and D_2 are connected to c .
2. For both values of $ab = 11$, $F = \bar{c}$. So D_3 is connected to $\bar{c}$.

S_1	S_0			
a	b	c	F	
0	0	0	0	$F = c$
0	0	1	1	
0	1	0	0	$F = c$
0	1	1	1	
1	0	0	0	$F = c$
1	0	1	1	
1	1	0	1	$F = \bar{c}$
1	1	1	0	

(a) Truth table



(b) Multiplexer implementation

Multiplexer



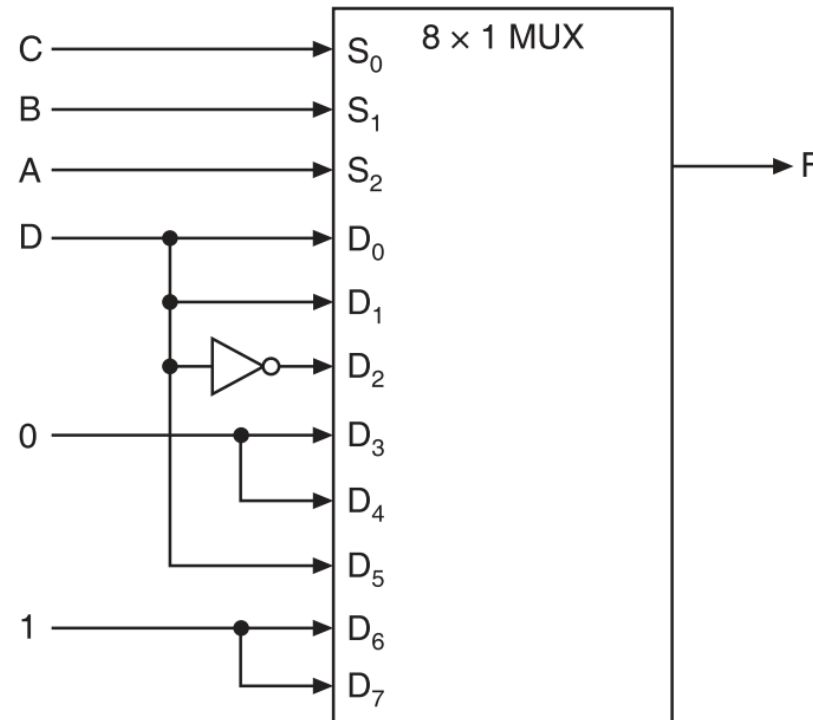
Implement the following logic function using an 8×1 MUX:

$$F(A, B, C, D) = \sum m(1, 3, 4, 11, 12, 13, 14, 15)$$

1. For both values of $ABC = 000$, $ABC = 001$ and $ABC = 101$, $F = D$. So D_0 , D_1 , and D_5 are connected to D .
2. For both values of $ABC = 010$, $F = \bar{D}$. So D_2 is connected to $\bar{D}$.
3. For both values of $ABC = 011$, and $ABC = 100$, $F = 0$. So D_3 and D_4 are connected to 0.
4. For both values of $ABC = 110$, and $ABC = 111$, $F = 1$. So D_6 and D_7 are connected to 1.

S_2	S_1	S_0		
A	B	C	D	F
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

(a) Truth table



(b) Logic diagram

Multiplexer

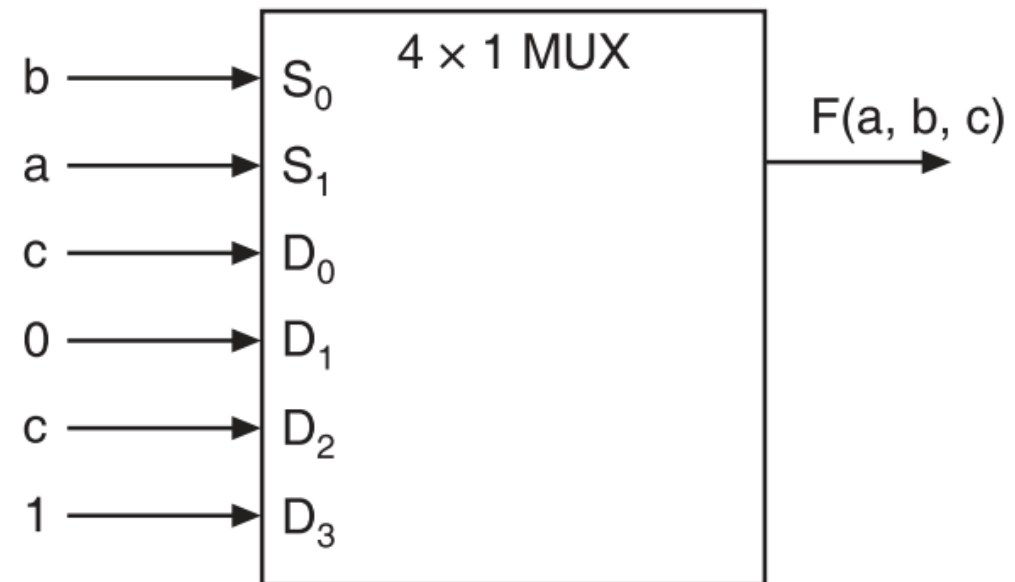


Implement the function $F(a, b, c) = ab + \bar{b}c$ using a 4 : 1 MUX.

$$\Sigma m(1, 5, 6, 7)$$

S_1	S_0	c	F	
a	b			
0	0	0	0	$F = c$
0	0	1	1	
0	1	0	0	$F = 0$
0	1	1	0	
1	0	0	0	$F = c$
1	0	1	1	
1	1	0	1	$F = 1$
1	1	1	1	

(a) Truth table



(b) Multiplexer implementation

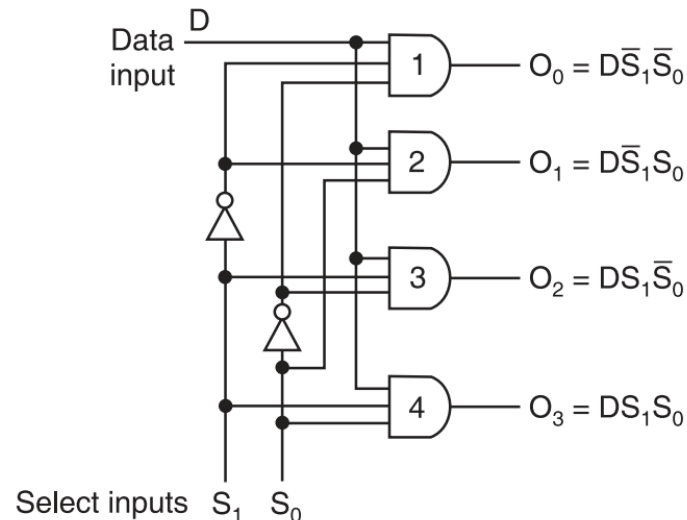
Demultiplexer



- ❖ A multiplexer takes several inputs and transmits one of them to the output.
- ❖ A demultiplexer performs the reverse operation; it takes a single input and distributes it over several outputs.
- ❖ So, a demultiplexer can be thought of as a ‘distributor’, since it transmits the same data to different destinations. Thus, whereas a multiplexer is an N -to-1 device, a demultiplexer is a 1-to- N (or 2^n) device.

1-Line to 4-Line Demultiplexer

- The input data line goes to all of the AND gates.
- The two select lines S_0 and S_1 enable only one gate at a time, and the data appearing on the input line will pass through the selected gate to the associated output line.

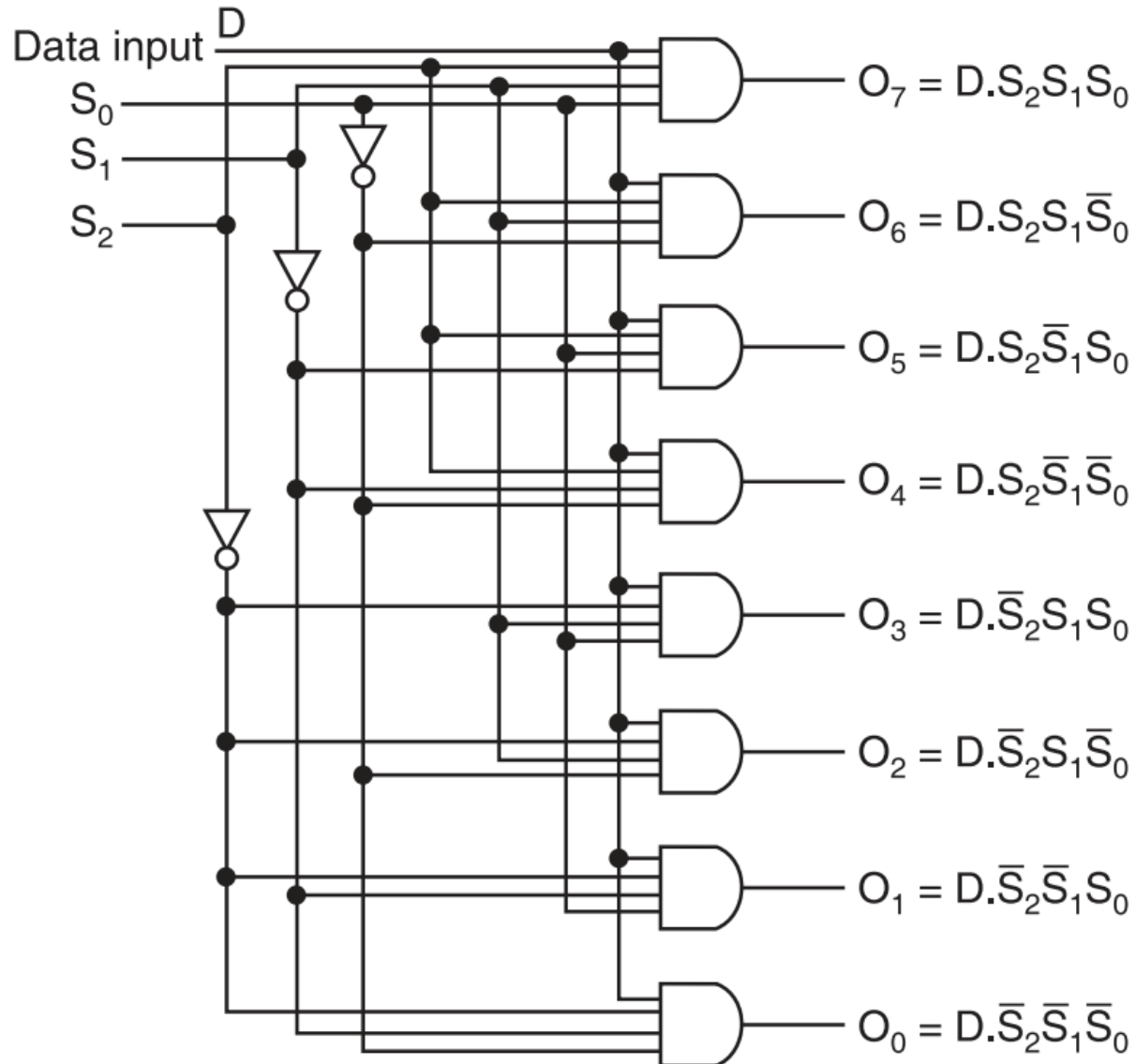


(a) Logic diagram

Select code		Outputs			
S_1	S_0	O ₃	O ₂	O ₁	O ₀
0	0	0	0	0	D
0	1	0	0	D	0
1	0	0	D	0	0
1	1	D	0	0	0

(b) Truth table

1-line to 8-line Demultiplexer



(a) Logic diagram

Select code			Outputs							
S_2	S_1	S_0	O_7	O_6	O_5	O_4	O_3	O_2	O_1	O_0
0	0	0	0	0	0	0	0	0	0	D
0	0	1	0	0	0	0	0	0	D	0
0	1	0	0	0	0	0	0	D	0	0
0	1	1	0	0	0	0	D	0	0	0
1	0	0	0	0	0	D	0	0	0	0
1	0	1	0	0	D	0	0	0	0	0
1	1	0	0	D	0	0	0	0	0	0
1	1	1	D	0	0	0	0	0	0	0

(b) Truth table

Decoder

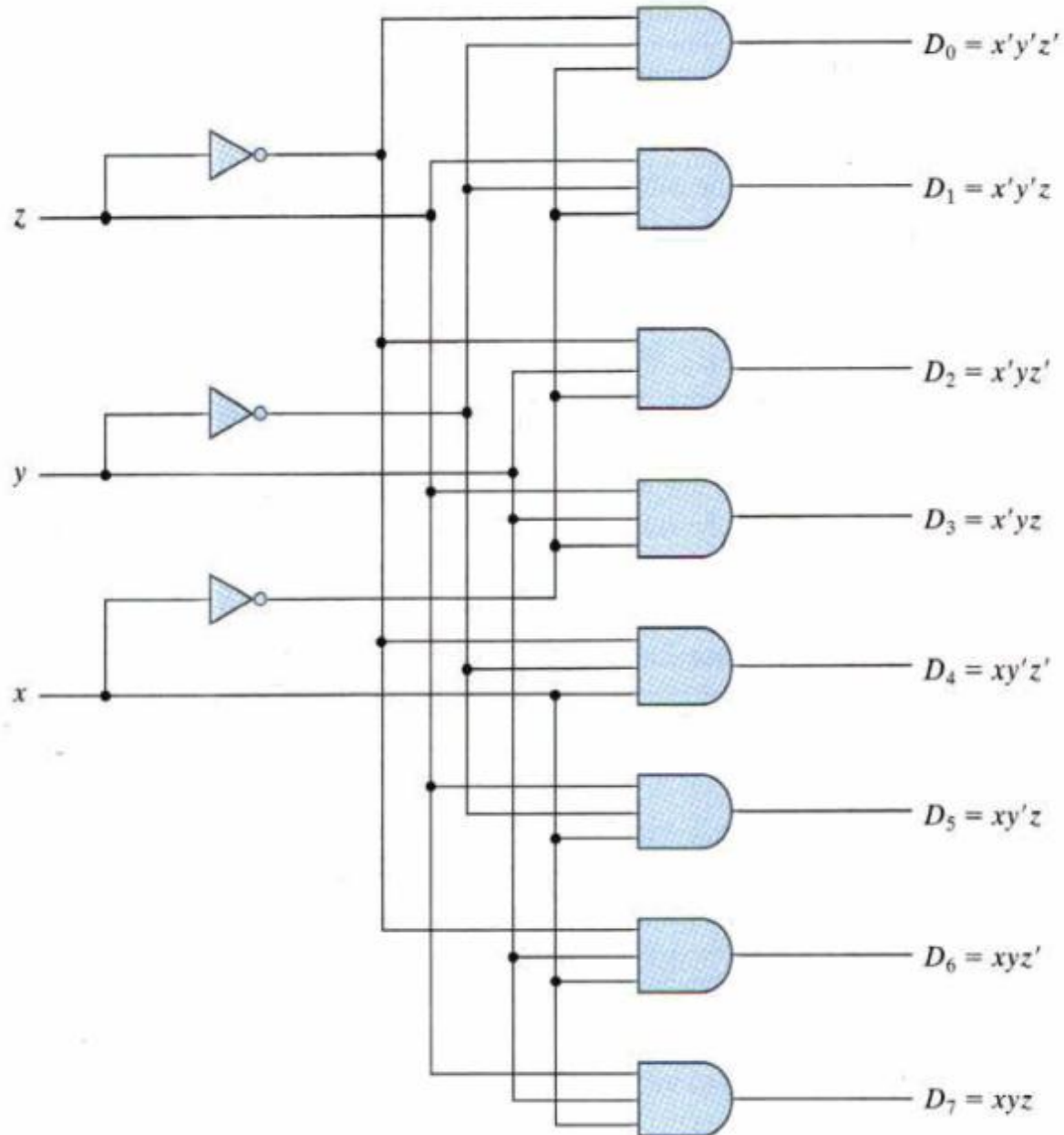


- ❖ A decoder in digital electronics is a combinational circuit **that converts binary input data into a unique output signal.**
- ❖ Converts binary information from n input lines to a maximum of 2^n unique output lines.
- ❖ The decoders are called n -to- m line decodes, where $m \leq 2^n$.

3-to-8 Line Decoder Circuit

- ❖ The three inputs are decoded into eight outputs, each representing one of the minterms of the three input variables.
- ❖ The three inverters provide the complement of the inputs, and each one of the eight AND gates generates one of the minterms.

3-to-8 Line Decoder Circuit



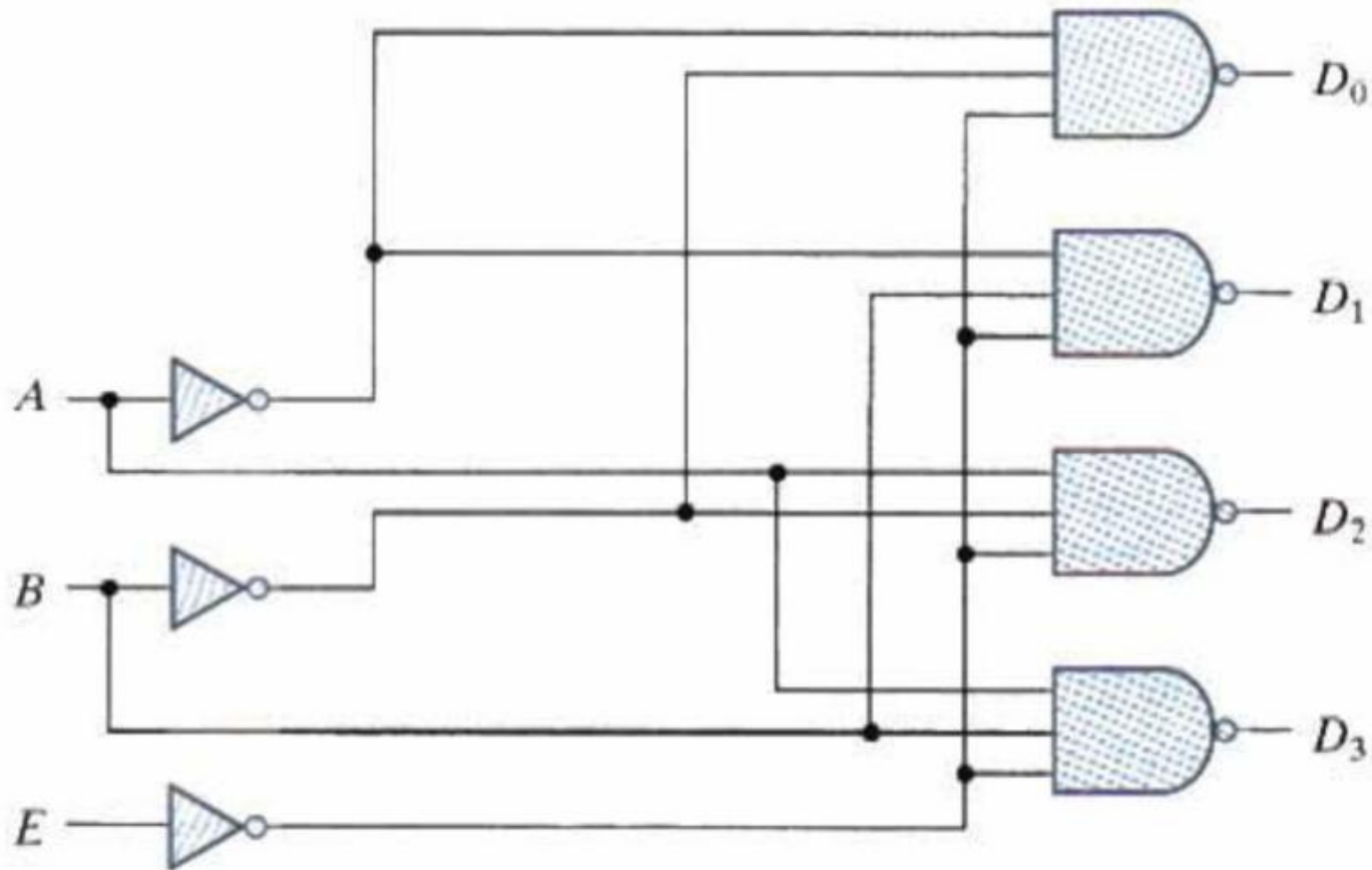
Truth Table of a Three-to-Eight-Line Decoder

Inputs			Outputs							
x	y	z	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

A particular application of this decoder is binary-to octal conversion.

The input variables represent a binary number, and the outputs represent the eight digits of a number in the octa number system.

2-to-4 Line Decoder with Enable Input

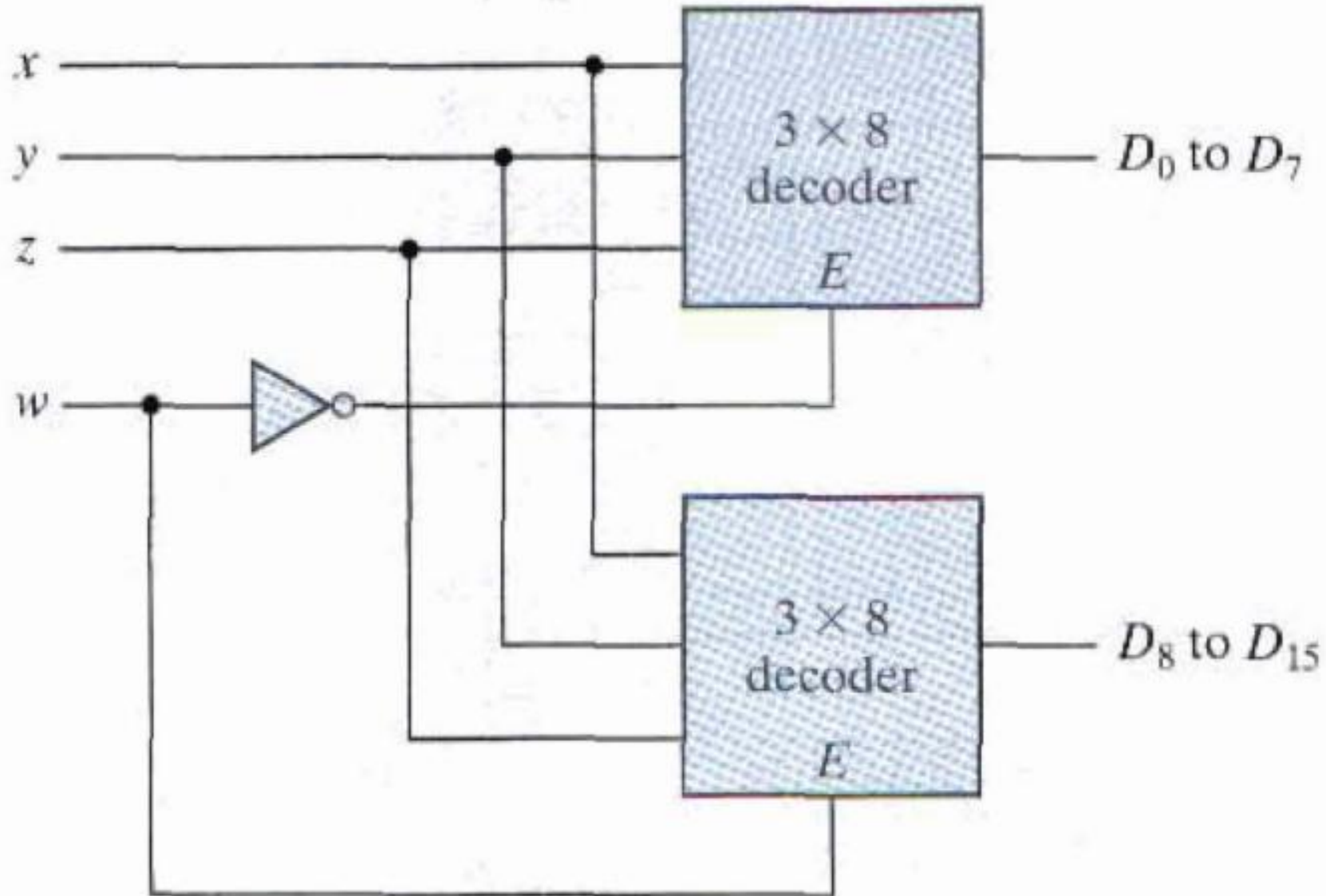


(a) Logic diagram

E	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

(b) Truth table

4-to- 16 Line Decoder



Implementation of Full Adder using Decoder

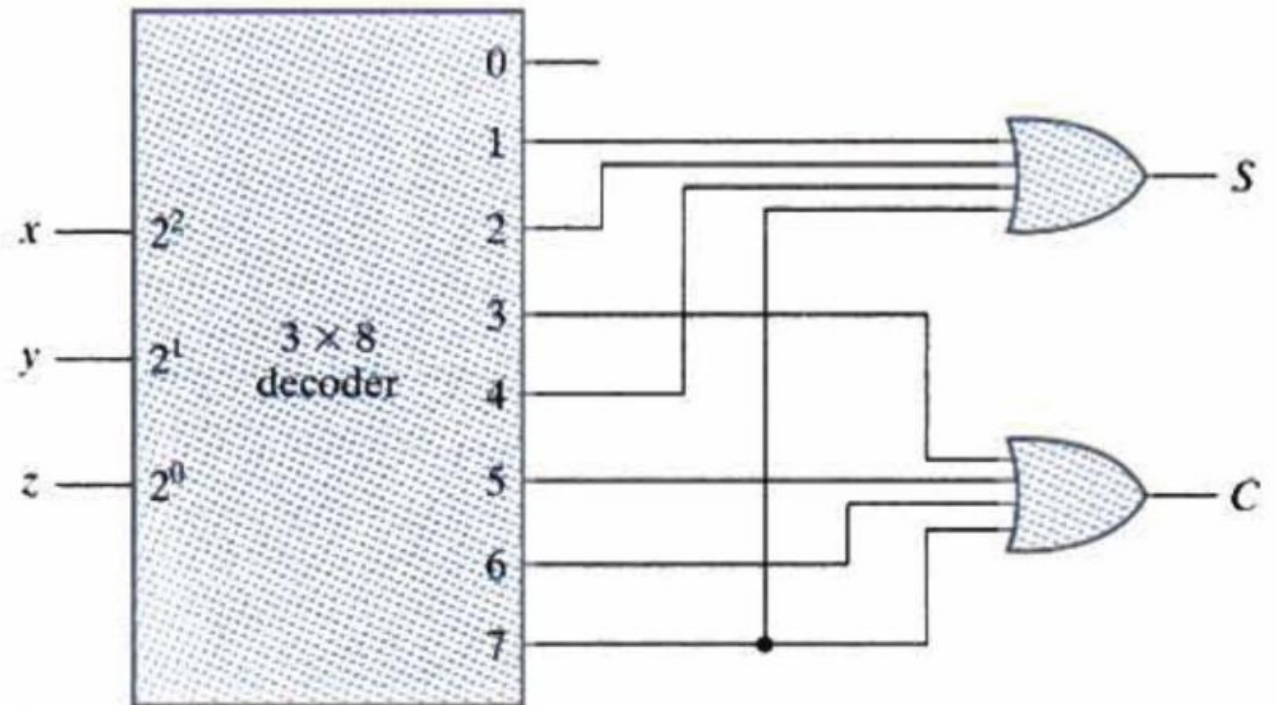


Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$S(x, y, z) = \Sigma(1, 2, 4, 7)$$

$$C(x, y, z) = \Sigma(3, 5, 6, 7)$$



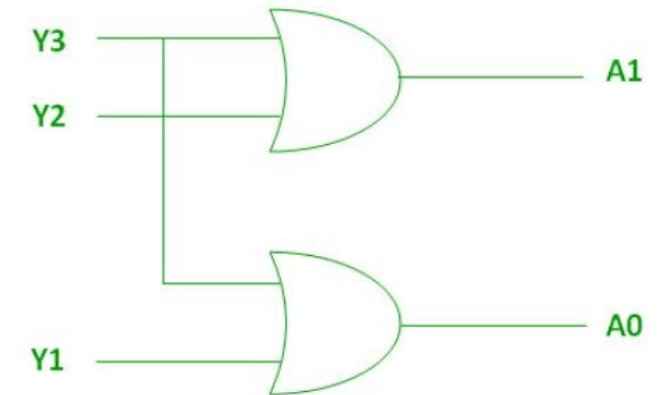
Encoder



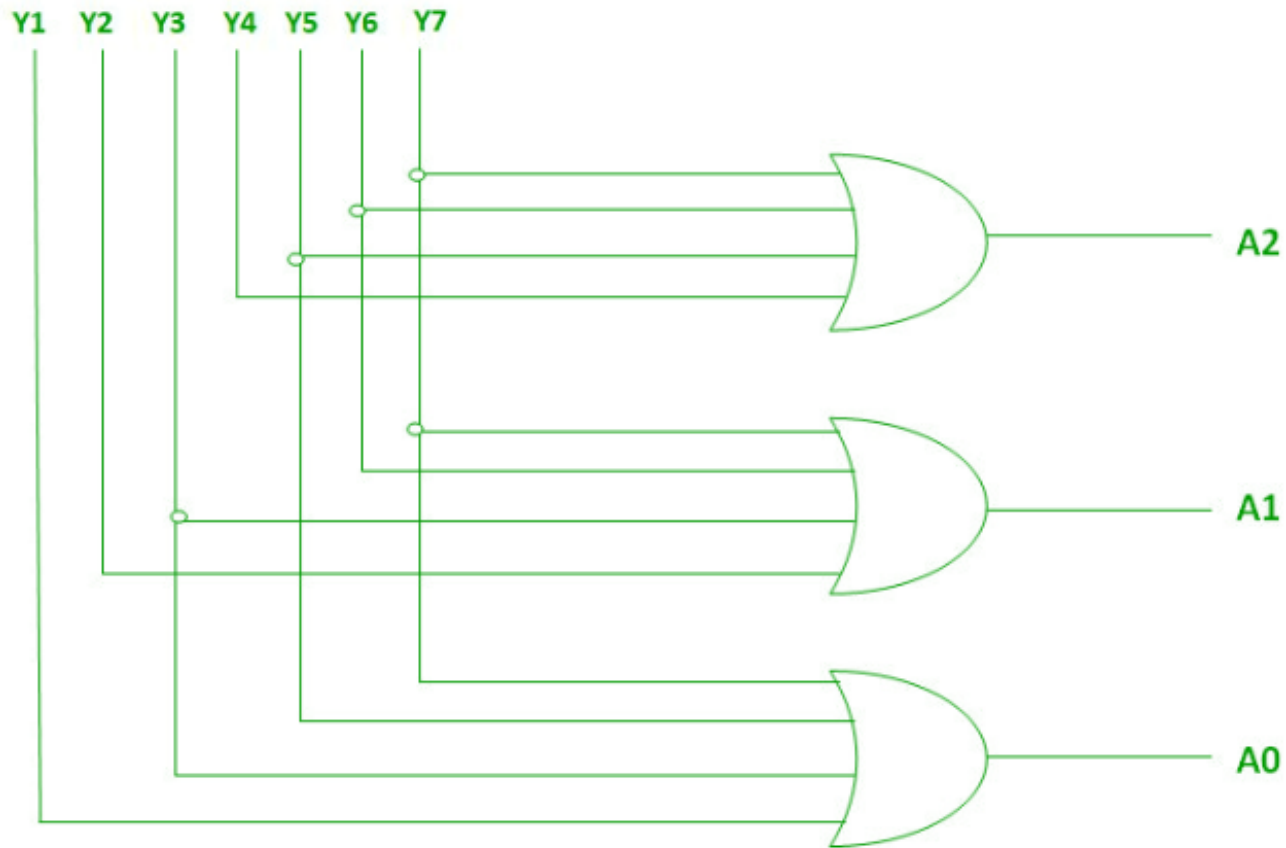
- ❖ An encoder is a digital circuit that performs the inverse operation of a decoder.
- ❖ An encoder has 2^n input lines and n output lines.
- ❖ The output lines, as an aggregate, generate the binary code corresponding to the input value.
- ❖ An example of an encoder is the octal-to-binary encoder.
- ❖ It has eight inputs (one for each of the octal digits) and three outputs that generate the corresponding binary number. It is assumed that only one input has a value of 1 at any given time.



INPUTS				OUTPUTS	
Y3	Y2	Y1	Y0	A1	A0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

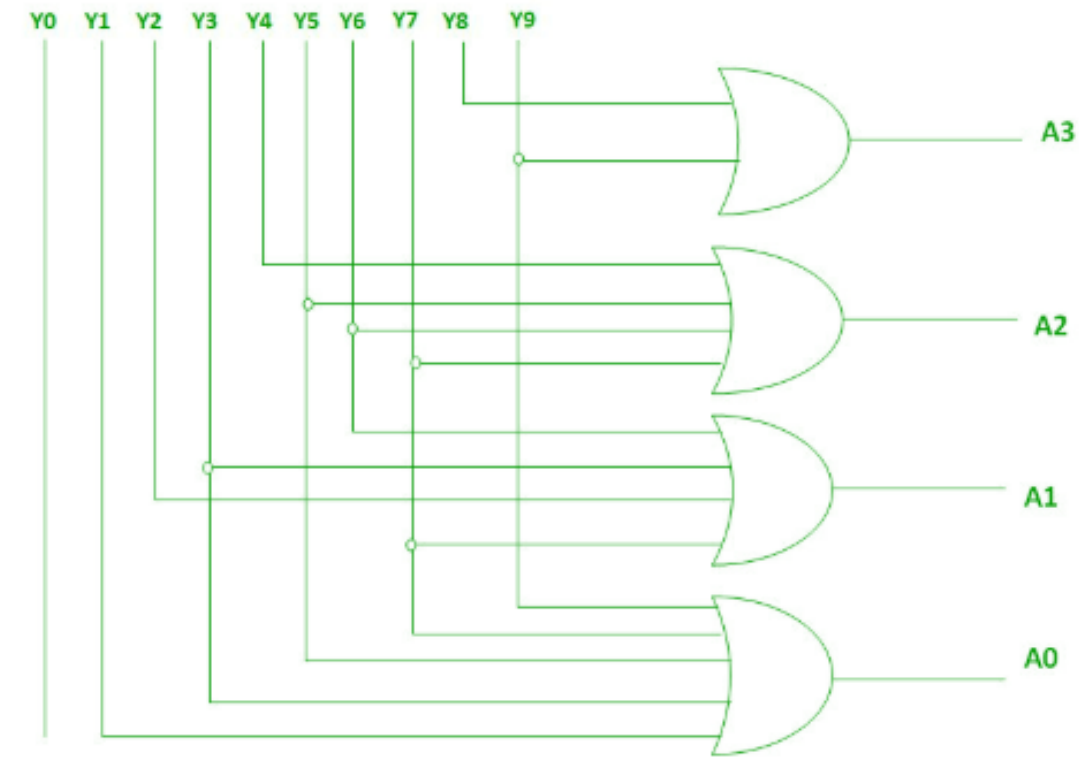
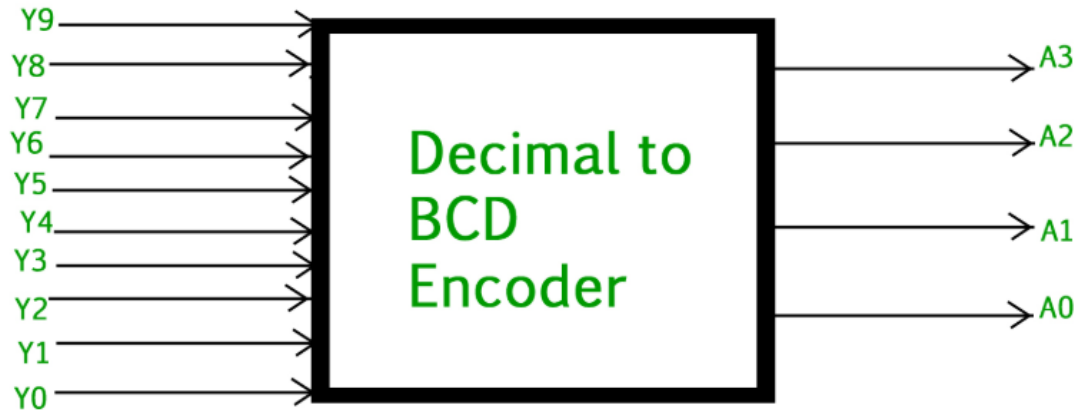


8 to 3 Encoder



INPUTS								OUTPUTS		
Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	A2	A1	A0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

Decimal to BCD Encoder

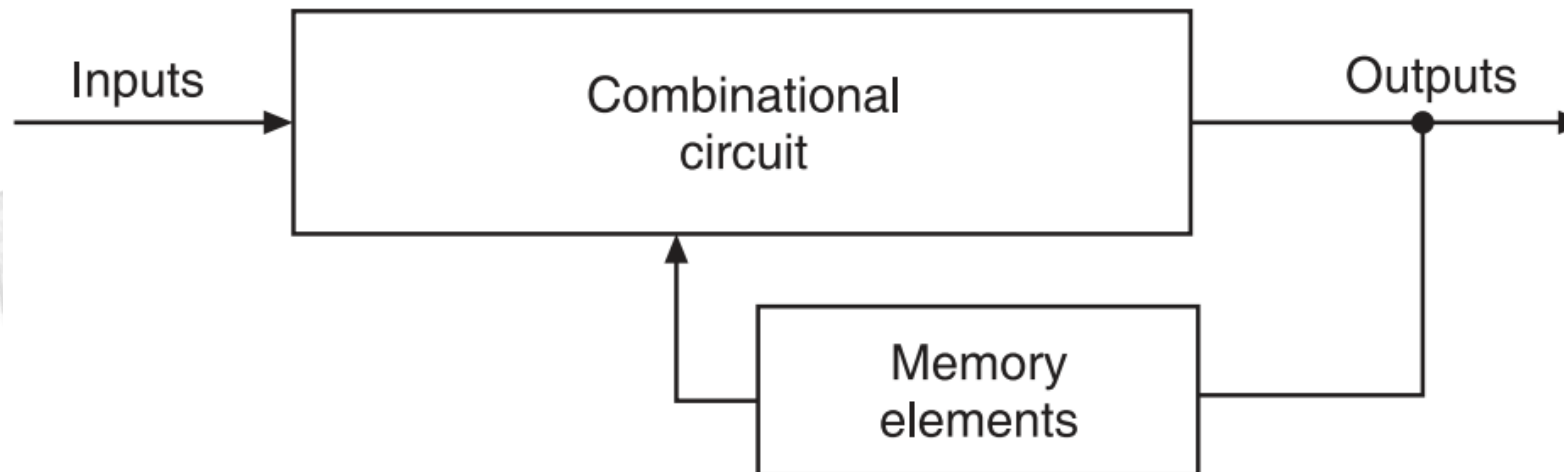


INPUTS										OUTPUTS			
Y9	Y8	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	A3	A2	A1	A0
0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	0	0	0	0	1	1	1
0	1	0	0	0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	0	1	0	0	1

Introduction to Sequential Circuits



- ❖ Sequential switching circuits are those whose output levels at any instant of time are dependent not only on the levels present at the inputs at that time, but also on the state of the circuit, i.e. on the prior input level conditions (i.e. on its past inputs).
- ❖ The past history is provided by feedback from the output back to the input.
- ❖ It means that sequential switching **circuits have memory**.
- ❖ **Sequential circuits are thus made of combinational circuits and memory elements. The past history is provided by feedback from the output back to the input.**



Introduction to Sequential Circuits



Combinational circuits

1. In combinational circuits, the output variables at any instant of time are dependent only on the present input variables.
2. Memory unit is not required in combinational circuits.
3. Combinational circuits are faster because the delay between the input and the output is due to propagation delay of gates only.
4. Combinational circuits are easy to design.

Sequential circuits

1. In sequential circuits, the output variables at any instant of time are dependent not only on the present input variables, but also on the present state, i.e. on the past history of the system.
2. Memory unit is required to store the past history of the input variables in sequential circuits.
3. Sequential circuits are slower than combinational circuits.
4. Sequential circuits are comparatively harder to design.

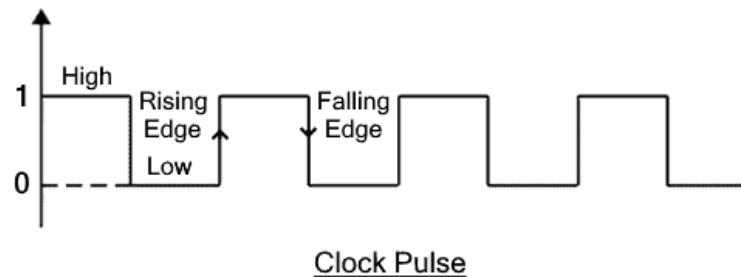
Classification of Sequential Circuits



- ❖ The sequential circuits may be classified as **synchronous sequential circuits** and **asynchronous sequential circuits** depending on the timing of their signals.
- ❖ **Periodically, recurring pulse is called a clock.**
- ❖ The sequential circuits which are controlled by a clock are called synchronous sequential circuits. **These circuits will be active only when clock signal is present.**
- ❖ The sequential circuits which are not controlled by a clock are called asynchronous sequential circuits, i.e. **the sequential circuits in which events can take place any time the inputs are applied** are called asynchronous sequential circuits.

What is a Clock Pulse?

A clock pulse is a periodic pulse signal that continuously oscillates between high state and low state. Usually, it is a square-type signal with a 50% duty cycle. Whereas the frequency of the clock may vary depending on the application.



A clock pulse has two levels i.e. **high level and low level**. It also has two edges i.e. **low to high (rising or positive edge)** and **high to low (falling or trailing or negative edge)**

Classification of Sequential Circuits

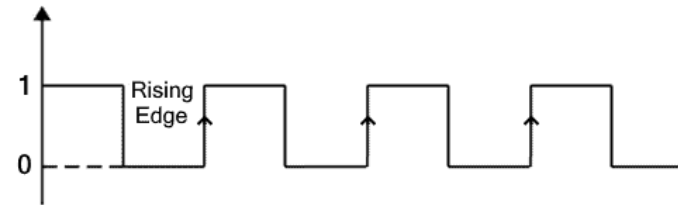


Edge Triggering

In edge triggering, the sequential circuit can change its state only when a clock edge occurs. Such sequential circuits are also called synchronous sequential circuits because they are synchronized with the edge of the clock. There are two types of edge-triggering

Positive Edge Triggering

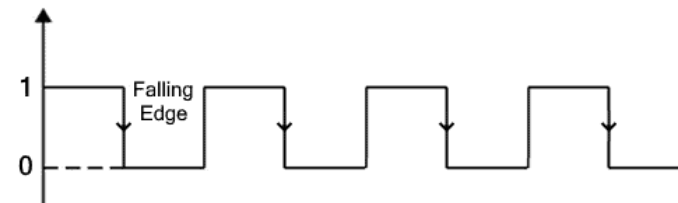
In positive edge triggering, the circuit only changes its state when there is a positive or rising edge at the clock input. The clock pulse goes from a low state to a high state. The circuit is said to be a positive edge-triggered circuit.



Positive Edge Triggering

Negative Edge Triggering

In negative edge triggering, the circuit only activates when there is a negative or falling clock edge. The pulse goes from a high to a low state. Such circuits are called negative or falling edge-triggered circuits.



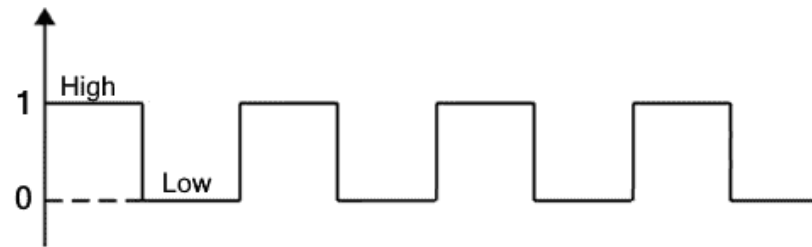
Negative Edge Triggering

Classification of Sequential Circuits



Level Triggering

In level triggering, the circuit is triggered during the high level or low level of the clock pulse. Its output changes when the clock level is maintained. Level triggering is divided into two types



Level Triggering

Positive Level Triggering

In positive-level triggering, the circuit is triggered only at the high level of the clock pulse.

Negative Level Triggering

In negative-level triggering, the circuit triggers at the low level of the clock pulse.

Storage Elements : Latches and Flip Flops



- ❖ A storage elements in a digital circuit can maintain a binary state indefinitely (as long as power is delivered to the circuit), until directed by an input signal to switch states.
- ❖ **Storage elements that operate with signal levels (rather than signal transitions) are referred to as latches; those controlled by a clock transition are flip flops.**
- ❖ Latches are said to be level-sensitive devices and flip flops are edge-sensitive devices.
- ❖ These two types of storage elements are related because latches are the basic circuits from which all the flip flops are constructed.